Updated GraphQL API Documentation

Endpoint: `http://localhost:3002/graphql` Playground: `http://localhost:3002/` (development only)

Authentication

```
{ "Authorization": "Bearer <token>" }
```

Queries

`me`

Get current authenticated user.

```
query GetMe {
  me { id fullName phoneNumber role isVerifiedTenant unitNumber }
}
```

`facilities`

Get all active facilities.

```
query GetFacilities {
  facilities { id name description pricePerHour openTime closeTime isActive
}
}
```

`facility`

```
query GetFacility($id: Int!) {
  facility(id: $id) { id name description pricePerHour openTime closeTime
isActive }
}
```

Variables: `{ "id": 1 }`

`availableSlots`

```
query GetAvailableSlots($input: SlotAvailabilityInput!) {
  availableSlots(input: $input) {
    facilityId
    date
    slots { startTime endTime isAvailable }
  }
}
```

Variables: `{ "input": { "facilityId": 1, "date": "2026-01-22" } }`

bookings 🆕

Get all bookings in the system. Admin only.

```
query GetBookings {
  bookings {
    id
    bookingDate
    startTime
    endTime
    status
    totalPrice
    user { id fullName }
    facility { id name }
  }
}
```

booking

```
query GetBooking($id: Int!) {
  booking(id: $id) {
    id bookingDate startTime endTime status totalPrice
    user { id fullName }
    facility { id name }
  }
}
```

Variables: `{ "id": 301 }`

userBookings

```
query GetUserBookings($userId: Int!) {
  userBookings(userId: $userId) {
    id bookingDate startTime endTime status totalPrice
    facility { name pricePerHour }
```

```
  }
}
```

Variables: `{ "userId": 3 }`

userDashboard

```
query GetUserDashboard($userId: Int!) {
  userDashboard(userId: $userId) {
    user { id fullName role }
    bookings { id bookingDate status facility { name } }
    bookingCountToday
    upcomingBookings
    totalSpent
  }
}
```

users 🆕

Get all users. Admin only.

```
query GetUsers {
  users { id fullName phoneNumber role isVerifiedTenant unitNumber }
}
```

user

```
query GetUser($id: Int!) {
  user(id: $id) { id fullName phoneNumber role isVerifiedTenant unitNumber
}
}
```

Mutations

login

```
mutation Login($input: LoginInput!) {
  login(input: $input) {
    token
    user { id fullName phoneNumber role isVerifiedTenant }
  }
}
```

Variables: `{ "input": { "phoneNumber": "+6281234567892" } }`

createFacility 🆕

```
mutation CreateFacility($input: CreateFacilityInput!) {
  createFacility(input: $input) {
    id name description pricePerHour openTime closeTime isActive
  }
}
```

Variables:

```
{
  "input": {
    "name": "Swimming Pool",
    "description": "Olympic-size indoor pool",
    "pricePerHour": 75000,
    "openTime": "06:00:00",
    "closeTime": "21:00:00",
    "isActive": true
  }
}
```

updateFacility 🆕

```
mutation UpdateFacility($id: Int!, $input: UpdateFacilityInput!) {
  updateFacility(id: $id, input: $input) {
    id name pricePerHour isActive
  }
}
```

Variables: `{ "id": 1, "input": { "pricePerHour": 80000, "isActive": false } }`

deleteFacility 🆕

```
mutation DeleteFacility($id: Int!) {
  deleteFacility(id: $id)
}
```

Variables: `{ "id": 1 }` **Response:** `{ "data": { "deleteFacility": true } }`

createUser 🆕

```
mutation CreateUser($input: CreateUserInput!) {
  createUser(input: $input) {
    id fullName phoneNumber role isVerifiedTenant unitNumber
  }
}
```

Variables:

```
{
  "input": {
    "fullName": "Siti Rahayu",
    "phoneNumber": "+6289876543210",
    "role": "TENANT",
    "isVerifiedTenant": true,
    "unitNumber": "0802"
  }
}
```

updateUser 🆕

```
mutation UpdateUser($id: Int!, $input: UpdateUserInput!) {
  updateUser(id: $id, input: $input) {
    id fullName role isVerifiedTenant unitNumber
  }
}
```

Variables: `{ "id": 3, "input": { "isVerifiedTenant": true, "unitNumber": "1203" } }`

deleteUser 🆕

```
mutation DeleteUser($id: Int!) {
  deleteUser(id: $id)
}
```

Variables: `{ "id": 3 }`

createBooking

```
mutation CreateBooking($input: CreateBookingInput!) {
  createBooking(input: $input) {
    id bookingDate startTime endTime status totalPrice
    user { fullName }
    facility { name }
```

```
    }
  }
```

Variables:

```
{
  "input": {
    "userId": 3,
    "facilityId": 1,
    "bookingDate": "2026-01-22",
    "startTime": "14:00:00",
    "endTime": "15:00:00"
  }
}
```

`confirmBooking`

```
mutation ConfirmBooking($id: Int!) {
  confirmBooking(id: $id) { id status }
}
```

`cancelBooking`

```
mutation CancelBooking($id: Int!) {
  cancelBooking(id: $id) { id status }
}
```

`deleteBooking` 🆕

Permanently removes the booking record. Admin only.

```
mutation DeleteBooking($id: Int!) {
  deleteBooking(id: $id)
}
```

Variables: `{ "id": 301 }` **Response:** `{ "data": { "deleteBooking": true } }`

Error Handling

```
{
  "errors": [{
```

```
    "message": "Error message",
    "extensions": { "code": "ERROR_CODE", "statusCode": 400 }
  }]
}
```

Error codes: `UNAUTHENTICATED`, `NOT_FOUND`, `VALIDATION_ERROR`, `BOOKING_CONFLICT`, `BOOKING_LIMIT_EXCEEDED`, `INVALID_TIME_SLOT`, `FACILITY_CLOSED`, `CONFLICT` (phone number already registered).

Testing GraphQL — the tool to use

Since you're already on Postman, the simplest answer is: **just use Postman**. It has full GraphQL support built in since 2020, works identically on Windows and Ubuntu, and you already know the interface. No new tool to learn.

Here's how to set it up:

1. Create a new request, change the method to `POST`, and set the URL to `http://localhost:3002/graphql`.

2. Go to the Body tab, select `GraphQL`. Postman will show two boxes — one for the query, one for the variables. It also fetches your schema automatically and gives you autocomplete.

3. For authenticated requests, go to the Headers tab and add:

```
Authorization    Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
```

4. Example — create a booking:

Query box:

```
mutation CreateBooking($input: CreateBookingInput!) {
  createBooking(input: $input) {
    id
    status
    totalPrice
    facility { name }
  }
}
```

Variables box:

```
{
  "input": {
```

```
      "userId": 3,
      "facilityId": 1,
      "bookingDate": "2026-03-20",
      "startTime": "10:00:00",
      "endTime": "11:00:00"
    }
  }
```

5. Save your requests into a Collection the same way you do for REST — one folder for Queries, one for Mutations, one for Auth. You can share the collection between your Windows and Ubuntu machines by exporting it as a JSON file or syncing through a Postman account.

The only alternative worth knowing about is **Insomnia**, which also runs on both Windows and Ubuntu and has a clean GraphQL mode, but since you're already in Postman there's no reason to switch.